CO326 Project Report

LED Luminaire Digital Twin Monitor

e20-co326-Digital-Twin-LED-Luminaire-Monitor

Department of Computer Engineering
University of Peradeniya
Sri Lanka

Contributors

E/20/032 — Bandara A.M.N.C.
E/20/157 — Janakanatha S.M.B.G.
E/20/254 — Methpura S.K.P.

April 2026

1. Problem Statement

Modern industrial and commercial facilities rely extensively on complex lighting networks for illumination. While various lighting technologies from traditional incandescent bulbs to modern light-emitting diodes (LEDs) offer different benefits in terms of efficiency and longevity, all are subject to degradation over time due to thermal, electrical, and environmental stresses. Unanticipated failures of these lighting systems can result in decreased productivity, safety hazards, and increased maintenance costs.

Currently, maintenance strategies are predominantly reactive (repairing after failure) or preventive (replacing on a fixed schedule), both of which are resource-inefficient. There is therefore a critical need for an intelligent, predictive monitoring system, a Digital Twin that can continuously track the health of any lighting fixture across a facility in real time, detect early signs of degradation such as intensity decay, colour shift, and electrical ripple, and accurately predict remaining useful life. Such a system would enable proactive maintenance, minimise downtime, and optimise the allocation of maintenance resources.

2. Chosen Parameters

The Digital Twin system relies on three primary parameters to track the health and degradation of each lighting fixture.

2.1 RGB Channels (R, G, B, and Derived Clear Channel)

RGB channel data is used as a proxy for monitoring spectral power distribution changes and colour shifts. As LED bulbs age, particularly due to thermal and optical stresses acting on their phosphors and physical packaging their chromaticity gradually drifts. Because colour instability often manifests before catastrophic failure occurs, monitoring the balance and trends among these channels serves as a vital early-warning indicator of health decline, even before overall light output drops significantly.

2.2 Intensity (Lumen Proxy via LDR and RGBC)

Intensity is the most direct and widely recognised metric for evaluating a luminaire's reliability, specifically for tracking lumen depreciation over time. It is modelled using exponential decay functions that account for accelerating factors such as elevated temperature, high humidity, and excessive drive current. This parameter links directly to the product's useful service life (following L70 standards) and represents the actual performance loss perceived by end users.

2.3 Electrical Ripple Percentage

While optical sensors capture the health of the light-emitting element itself, the ripple percentage monitors the condition of the electrical driver circuit. Over time, electrolytic capacitors within the driver degrade, causing an increase in equivalent series resistance (ESR) and a corresponding rise in output current ripple. By tracking this parameter, the system gains a complete, system-level picture of health, detecting electrical stress pathways and driver deterioration that optical-only sensing would otherwise miss.

3. Solution Architecture

The system is structured as a two-tier architecture that combines edge-level intelligence with cloud-level analytics. At the edge, ESP32 microcontrollers equipped with LDR and RGBC sensors perform real-time data collection and on-device TinyML inference. Collected telemetry is published via MQTT to a centralised Mosquitto broker. A Node-RED instance routes incoming messages into an InfluxDB time-series database and exposes a local dashboard. A Python-based Cloud AI service queries InfluxDB, performs feature engineering, and runs XGBoost models to generate failure probability predictions and alerts. Grafana dashboards provide the primary user interface for facility managers, visualising real-time health metrics, ML predictions, and alert histories.

4. Simulation Implementation

To thoroughly test the predictive machine learning models and the data infrastructure without waiting years for natural luminaire failures, the project employs a dual-simulation approach: a Software Digital Twin Simulator for mathematical modelling, and a Physical Hardware Simulator for sensor validation.

Simulation window

4.1 Software Digital Twin Simulator

The software simulator is a full-stack application comprising a Node.js backend and a Vue.js frontend, designed to generate highly realistic, accelerated telemetry.

Mathematical Degradation Models

The core simulation engine models the decay of a luminaire using physics-inspired formulas. Lumen depreciation is modelled using an exponential decay function (exp(-k*t)), which is further accelerated by environmental stress factors such as elevated ambient temperature, high humidity, and excessive drive current.

Multivariate Output

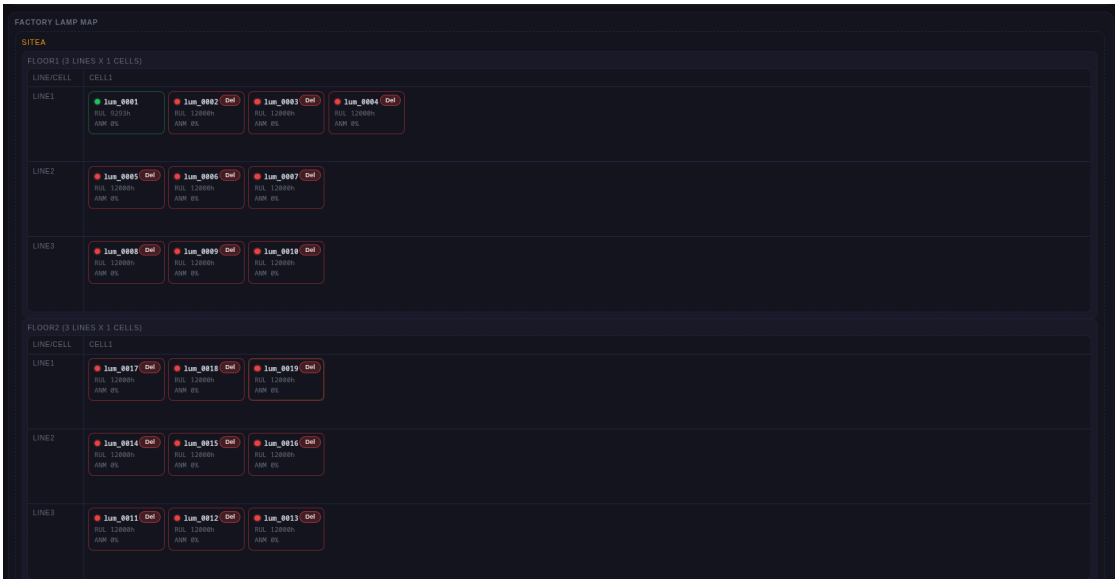
The simulator does not output intensity alone; it models complex, realistic field failures. RGB colour shift is simulated by altering channel ratios to mimic phosphor degradation over time, and electrical ripple percentage is modelled to represent the ageing of electrolytic capacitors within the LED driver.

Interactive UI and Fault Injection

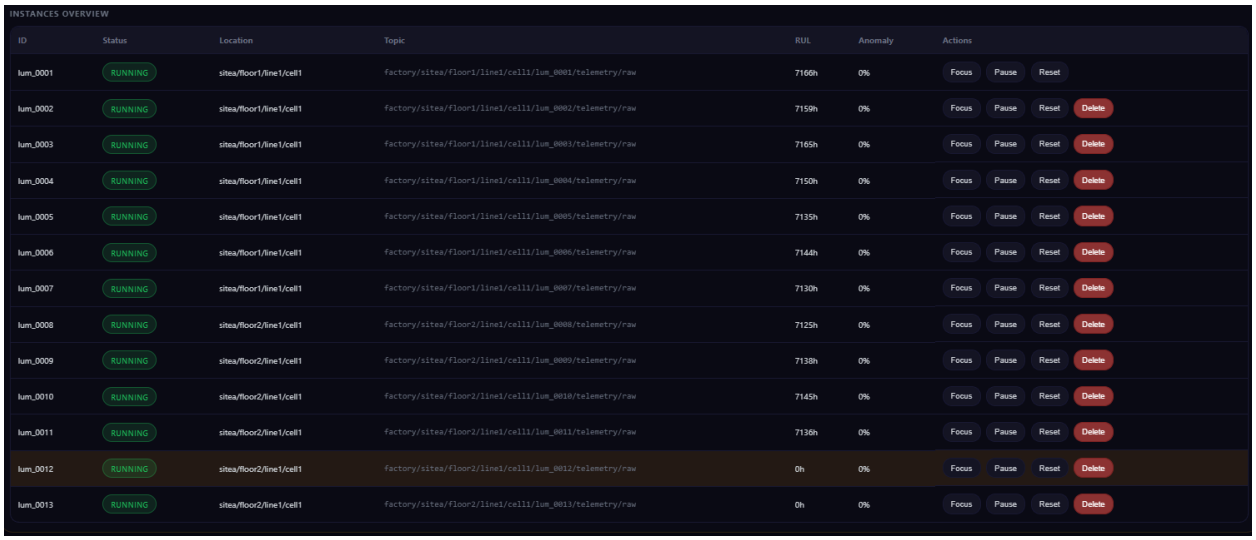
Fault injection

The Vue.js frontend provides interactive charts and real-time controls, allowing manipulation of environmental variables and inject specific anomalies such as sudden phosphor failure or large ripple spikes in order to observe how the AI models respond. The generated data is packaged as JSON and streamed directly into the MQTT broker.

Multiple Instances Generation



Multiple instances generation



Multiple instances running

Beyond simulation accuracy, its key strength is multi-instance scaling where each luminaire runs as an independent stateful instance with its own environmental conditions, anomaly flags, and RUL trajectory, and publishes structured JSON over hierarchical MQTT topics (factory/{site}/{floor}/{line}/{cell}/{luminaireId}/...). This enables realistic high-volume streaming into Node-RED, time-series storage in InfluxDB, and Grafana dashboards for per-device and fleet-level monitoring, while also feeding both Edge AI and Cloud AI workflows for robust predictive maintenance validation.

4.2 Physical Hardware Simulator

The physical hardware simulator validates the Edge AI firmware and optical sensors under realistic conditions.

Implementation

Built using an Arduino Uno (led_sim.ino), this simulator acts as the physical light source for the ESP32's Light Dependent Resistor (LDR). It compresses the entire multi-year lifespan of an LED bulb into a configurable, repeating one-to-thirty-second window.

Phase Mapping via PWM

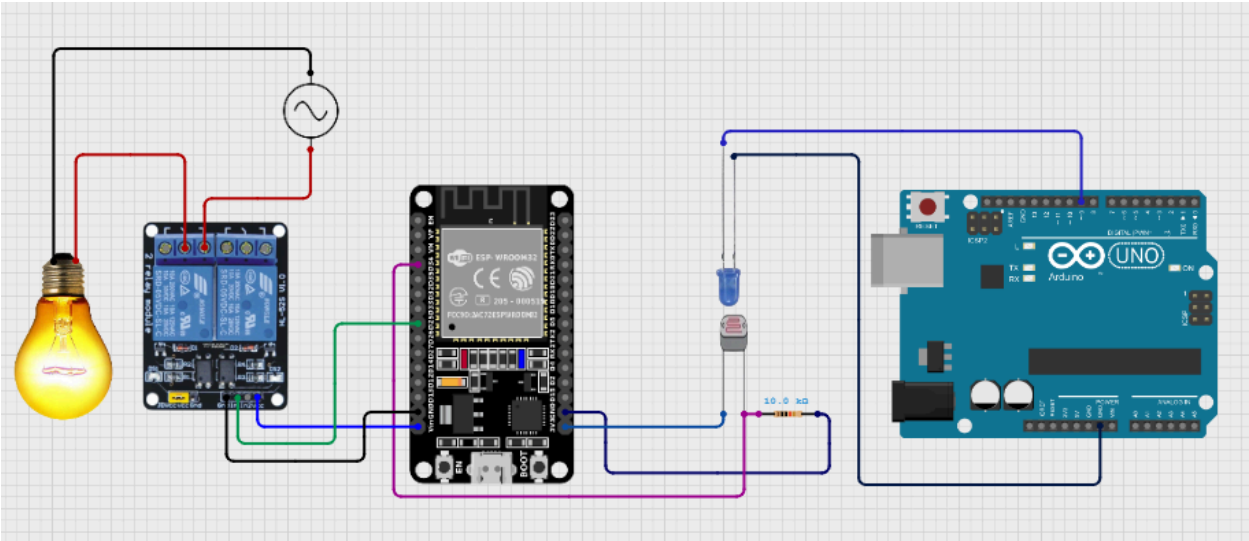
The Arduino uses precise Pulse Width Modulation (PWM) manipulation to mimic the three operational phases recognised by the TinyML model:

- HEALTHY — A smooth, high-intensity output representing the stable early life of the bulb.
- DEGRADING — A gradually declining duty cycle with random noise to create a slow flicker effect.
- FAILED — A low-intensity, rapid strobe pattern to represent end-of-life behaviour.

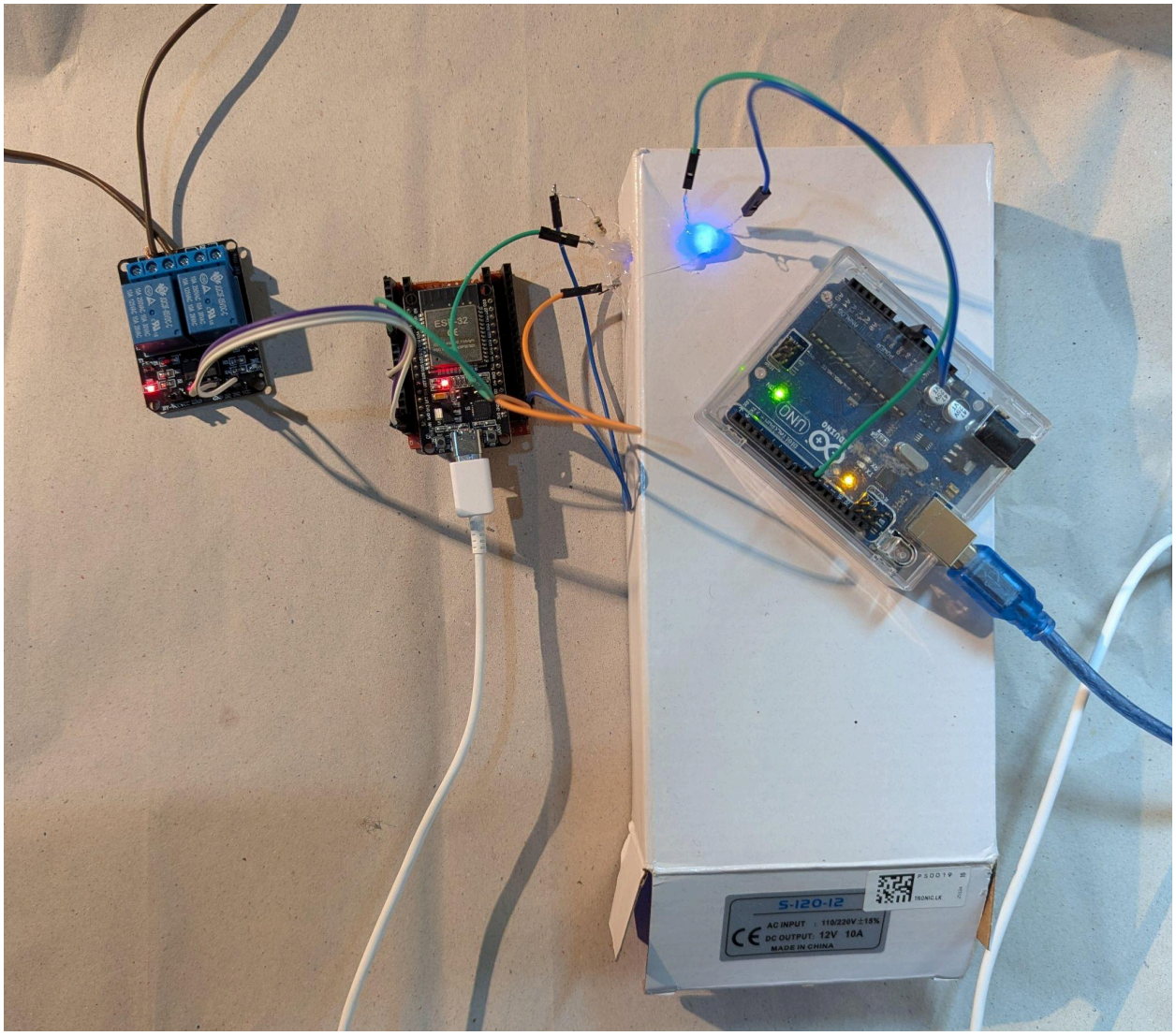
This physical simulation provides the edge node with authentic, fluctuating optical signals to classify in real time.

5. Hardware Implementation

The hardware architecture of the Digital Twin monitoring system is designed to be lightweight, cost-effective, and capable of executing machine learning models directly at the edge. The implementation is divided into the primary Edge AI monitoring node and the physical lifecycle simulator.



Hardware implementation (ldr, relay and esp32) and simulator using arduino uno



Real setup

5.1 Core Microcontroller — ESP32

The system uses an ESP32 DevKit (30-pin) as the central processing unit for the edge nodes. The ESP32 was selected for its dual-core architecture, built-in Wi-Fi capabilities, and sufficient memory (using the "Huge APP" partition scheme) to host and execute TensorFlow Lite Micro models locally. This allows each node to perform real-time data collection, run TinyML inference, and publish MQTT telemetry simultaneously without relying on continuous cloud connectivity.

5.2 Sensor Configuration

A Light Dependent Resistor (LDR) is used as the primary optical sensor for monitoring light intensity.

- Circuitry: The LDR is configured in a voltage divider circuit with a 10 kΩ pull-down resistor connected to ground.
- Interfacing: The output voltage is fed into the ESP32's analogue pin (GPIO 34 / ADC1_CH6). The ESP32's 12-bit Analogue-to-Digital Converter (ADC) reads voltage

variations caused by changing light levels, yielding high-resolution raw values from 0 to 4095 that are fed into the feature extraction sliding window for the AI model.

5.3 Actuation and Relay Control

To provide a fail-safe switching mechanism, the ESP32 interfaces with a single-channel relay module.

- Connection: The relay is connected to GPIO 26 and operates on active-LOW logic.
- Logic: During the HEALTHY state, the relay remains OFF. When the TinyML model detects a DEGRADING or FAILED state, the ESP32 triggers the relay to switch ON, which activates a backup lighting circuit or triggers a localised physical alarm on the factory floor.

5.4 Physical Hardware Simulator — Arduino Uno

A separate hardware simulator was built using an Arduino Uno (led_sim.ino) to validate the system physically.

- Setup: An LED with a current-limiting resistor is connected to a PWM-capable pin (Pin 9) on the Arduino.
- Simulation Physics: The Arduino compresses a bulb's entire lifespan into a configurable one-to-thirty-second window. By manipulating the PWM duty cycle, it physically mimics the optical behaviour of a degrading bulb across the HEALTHY, DEGRADING, and FAILED phases, providing a realistic, fluctuating light source for the ESP32's LDR to monitor.

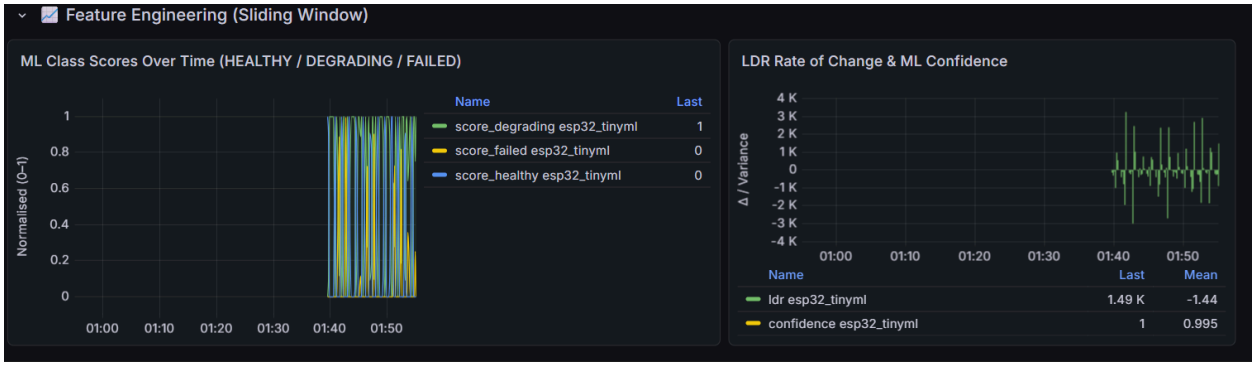
6. Edge AI Implementation

The Edge AI firmware (tinyml_light_monitor.ino) was developed using the EloquentTinyML v3.x wrapper and TensorFlow Lite Micro (tflm_esp32). By running machine learning inference directly on the ESP32 microcontroller, the system eliminates the round-trip latency that cloud-based classification would incur, enabling truly real-time responses.

6.1 Feature Engineering at the Edge

A sliding window approach with a window size of 20 samples is implemented on the ESP32 to extract six real-time features from raw LDR ADC readings:

- ldr_norm — Normalised current LDR reading.
- avg_norm — Normalised moving average of LDR readings across the window.
- rate_of_change — First-order derivative of the LDR signal.
- variance — Within-window variance of the LDR signal.
- min_norm — Normalised minimum reading in the window.
- max_norm — Normalised maximum reading in the window.

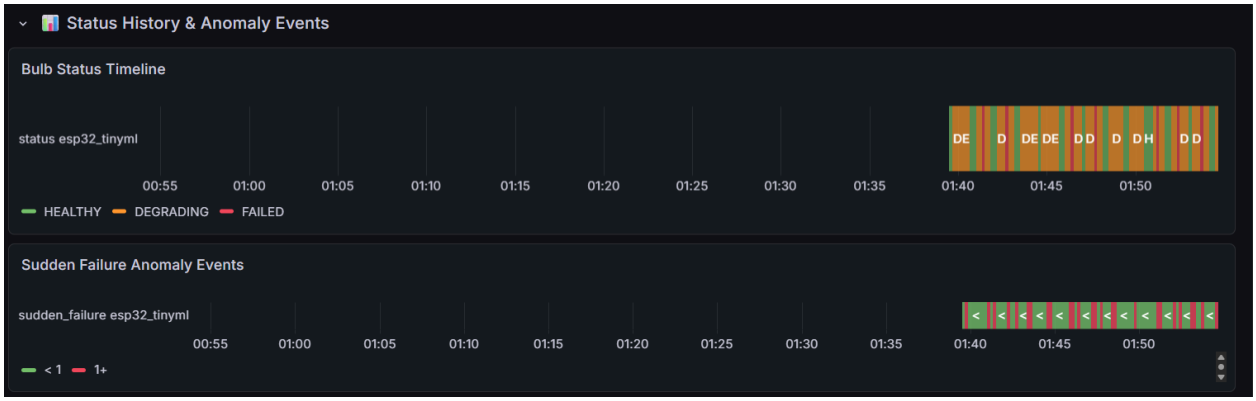


Feature enginnering

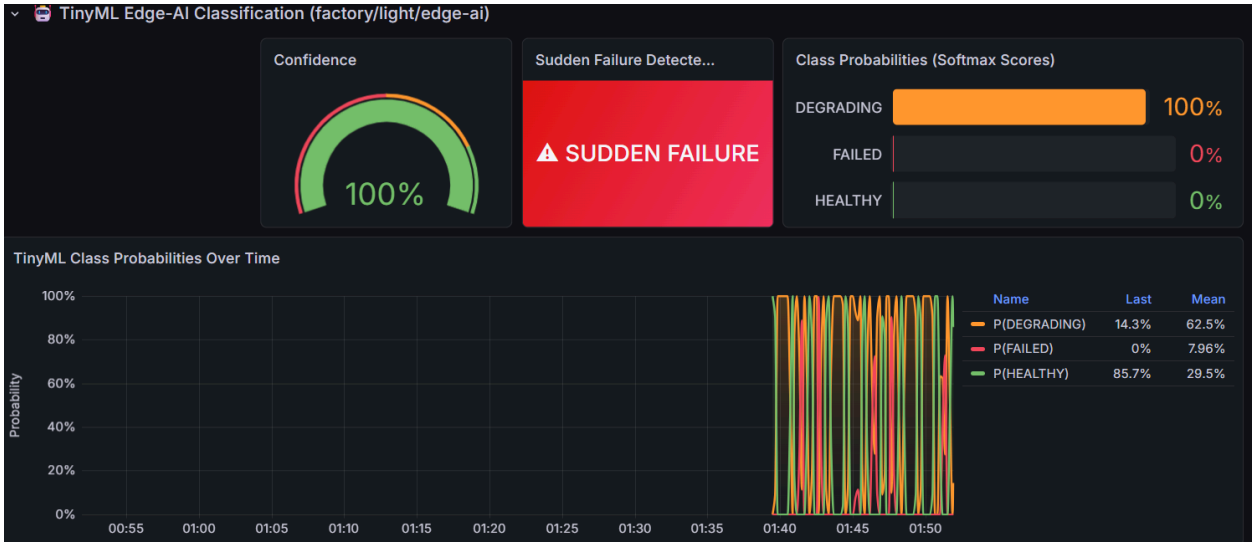
6.2 Anomaly Detection and Inference Logic

Two complementary detection mechanisms are deployed on the edge device:

- Z-Score-Based Sudden Failure Detector: A statistical algorithm that triggers an immediate MQTT alert when the current LDR reading deviates more than three standard deviations from the moving average, enabling near-instant detection of abrupt failures.
- TinyML Neural Network Classifier: A neural network with Dense layers and a Softmax output is deployed directly on the ESP32 to classify the bulb state into HEALTHY, DEGRADING, or FAILED, along with calibrated confidence scores for each class.



History



Anomaly detection

6.3 TinyML Model Architecture

The Keras neural network trained for ESP32 deployment has the following architecture:

- Input layer: 6 features (ldr_norm, avg_norm, rate_of_change, variance, min_norm, max_norm).
- Hidden Layer 1: Dense(16, ReLU activation).
- Hidden Layer 2: Dense(8, ReLU activation).
- Output layer: Dense(3, Softmax) — outputs probabilities for HEALTHY, DEGRADING, and FAILED classes.
- Target model size: approximately 2 KB after int8 quantisation.
- Export format: TFLite → bulb_model.h C header file for inclusion in the Arduino sketch.

Training data is generated by the software simulator's degradation engine, with varied environmental conditions (temperature 15–45 °C, drive current 200–500 mA, humidity 20–80%), producing approximately 100,000 labelled samples. Labels are assigned as: HEALTHY (intensity > 0.7), DEGRADING (0.3 ≤ intensity ≤ 0.7), and FAILED (intensity < 0.3).

6.4 Telemetry Publishing

ML predictions, confidence scores, and raw LDR values are formatted as JSON payloads and published via MQTT to the factory/light/edge-ai topic, from where they are consumed by both Node-RED and the Python Cloud AI service.

7. Cloud AI Implementation

The server-side intelligence is implemented as a Python microservice (python-edge) that runs within the Docker stack. It handles time-horizon failure prediction, comprehensive anomaly detection, and automated alert generation — tasks that require access to historical telemetry and are too computationally intensive for the ESP32.

7.1 Service Architecture

The Python service exposes a FastAPI interface on port 5000. It subscribes to MQTT telemetry topics, periodically queries InfluxDB for historical sensor data, and exposes a REST API that can be called directly for on-demand predictions. The primary prediction endpoint accepts POST requests to /api/predict, which require full sensor readings including LDR, temperature, drive current, voltage, humidity, RGB channels, power consumption, and ripple percentage.

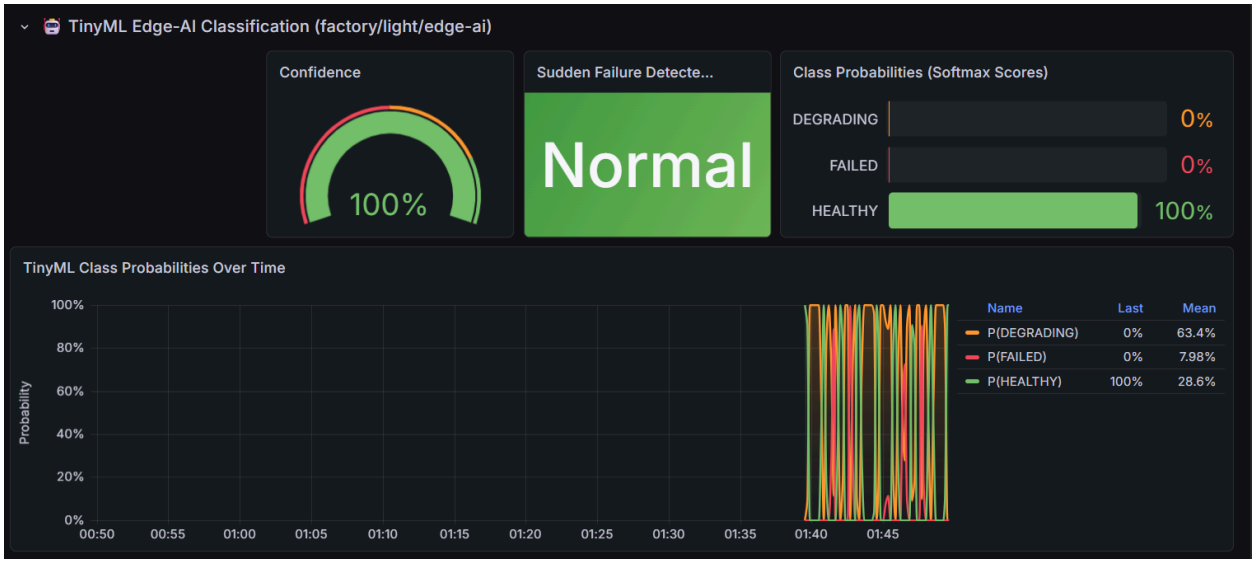
7.2 Feature Engineering

The feature engineering module (app/feature_engine.py) transforms raw time-series sensor readings into meaningful indicators for the XGBoost models. Engineered features include: rolling mean brightness over 7-day and 14-day windows, brightness slope and variance, the maximum drop in a 24-hour period, cumulative operating hours, mean ripple percentage, colour shift expressed as R/G ratio drift, hours spent above 25 °C, and drive current standard deviation.

7.3 Machine Learning Models

Two distinct XGBoost models are deployed in the cloud tier:

- Fault Classifier (app/fault_classifier.py): Trained on a 34,000-row street light fault prediction dataset (street_light_fault_prediction_dataset.csv), this model classifies the current fault type into one of five categories.
- Failure Predictor (app/predictor.py): A calibrated XGBoost classifier that outputs failure probabilities for three prediction horizons — 3 days, 7 days, and 14 days — along with an estimated number of remaining days and a confidence score.



TinyML data represent in grafana

Failure prediction

Failure probability

7.4 Alert Engine

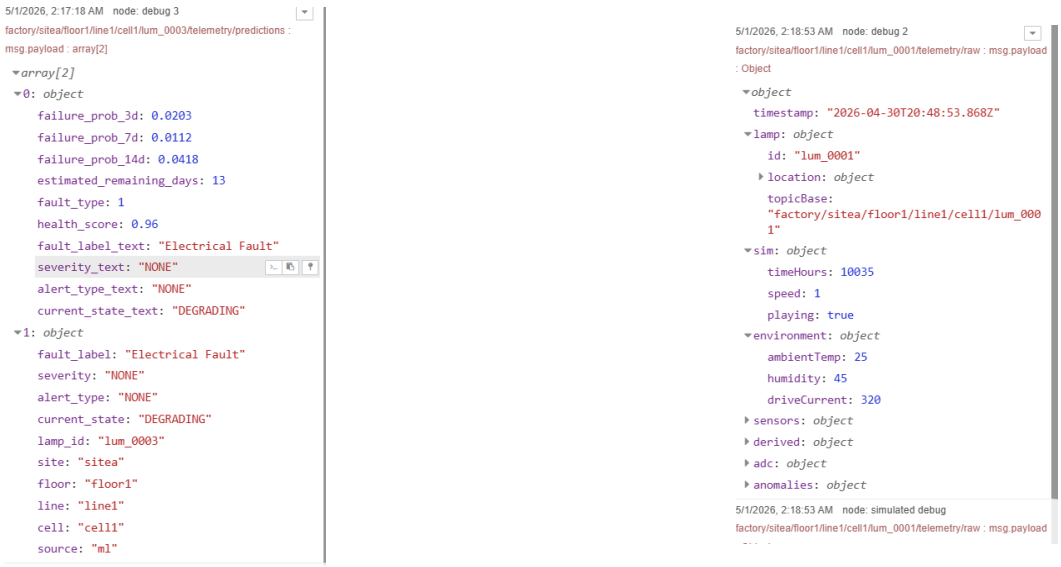
Once failure probabilities are computed, the alert engine (app/alert_engine.py) maps them to four severity levels and publishes structured alert messages to the MQTT alerts topic:

Severity	Trigger Condition
INFO	P(14-day failure) between 30% and 50%
MEDIUM	P(14-day failure) between 50% and 70%
HIGH	P(14-day failure) > 70%, or P(7-day failure) > 50%
CRITICAL	Sudden failure detected, or P(3-day failure) > 70%

Show alert in the multi bulb system. Users can easily identify the fault bulbs

7.5 MQTT Topic Structure

Topic	Publisher	Payload Summary
factory/.../telemetry/raw	ESP32 / Simulator	Full sensor reading (all fields required)
factory/.../telemetry/edge-ai	ESP32	{"status":"HEALTHY","confidence":0.92,"sudden_failure":false}
factory/.../alerts/status	Python service	{"severity":"HIGH","probability_14d":0.73,"message":"..."}



Mqtt topics payload in Node-red

8. System Setup

The following steps describe the complete procedure for deploying the LED Luminaire Digital Twin Monitor from scratch. The project repository is available at:

https://github.com/cepdnaclk/e20-co326-Digital-Twin-LED-Luminaire-Monitor

Step 1: Install Training Dependencies

Ensure Python 3.9 or later is installed, then install the required packages:

```
pip install tensorflow numpy pandas scikit-learn xgboost joblib scipy
```

Step 2: Run the Training Pipeline

Execute the following scripts in order from the ml/training/ directory:

```
cd ml/training
python generate_tinyml_training_data.py
python train_tinyml_classifier.py
python generate_synthetic_data.py
python train_fault_classifier.py
python train_failure_predictor.py
```

Step 3: Flash the ESP32 Firmware

1. Open edge-logic/firmware/light_monitor/tinyml_light_monitor.ino in the Arduino IDE.
2. Install the required Arduino libraries: EloquentTinyML and PubSubClient.
3. Confirm that bulb_model.h (exported by train_tinyml_classifier.py) exists in the same directory as the .ino file.
4. Upload the sketch to the ESP32 and verify the output on the Serial Monitor. The device should display class labels (HEALTHY / DEGRADING / FAILED) and confidence scores.

Step 4: Start the Docker Stack

All backend services are orchestrated via Docker Compose:

```
cd docker
docker compose up -d --build
```

This command starts the following containerised services: Mosquitto MQTT broker, InfluxDB, Grafana, Node-RED, the Node.js/Vue.js simulator, and the Python Edge AI service.

Step 5: Verify the System

Confirm that each component is operating correctly:

```
# Check Python Edge AI service health
curl http://localhost:5000/api/health

# Test a prediction endpoint
curl -X POST http://localhost:5000/api/predict \
  -H "Content-Type: application/json" \
  -d '{"bulb_id":"lum_0001","readings":[{"timestamp":
  "2026-04-29T12:00:00Z","ldr":3500,"temperature":25.0,
  "current":320.0,"voltage":230.0,"humidity":45.0,
  "rgb":{"r":255,"g":240,"b":225},
  "power_consumption":60.0,"ripple_percent":1.6}]}'
```

Step 6: Access the Dashboards

- Grafana is accessible at http://localhost:3000. The default credentials are admin / admin.
- Node-RED flows can be managed at http://localhost:1880.
- The Python AI service REST API is available at http://localhost:5000.

9. Model Accuracy

The following accuracy targets were defined during the design phase and are used as acceptance criteria for the trained models:

Model	Metric	Target
TinyML Classifier (ESP32)	Classification Accuracy	> 85% on held-out test set; model size < 5 KB
Fault Classifier (XGBoost)	Classification Accuracy	> 80% on the street light dataset
Failure Predictor (XGBoost)	AUC-ROC	> 0.75 on held-out sequences

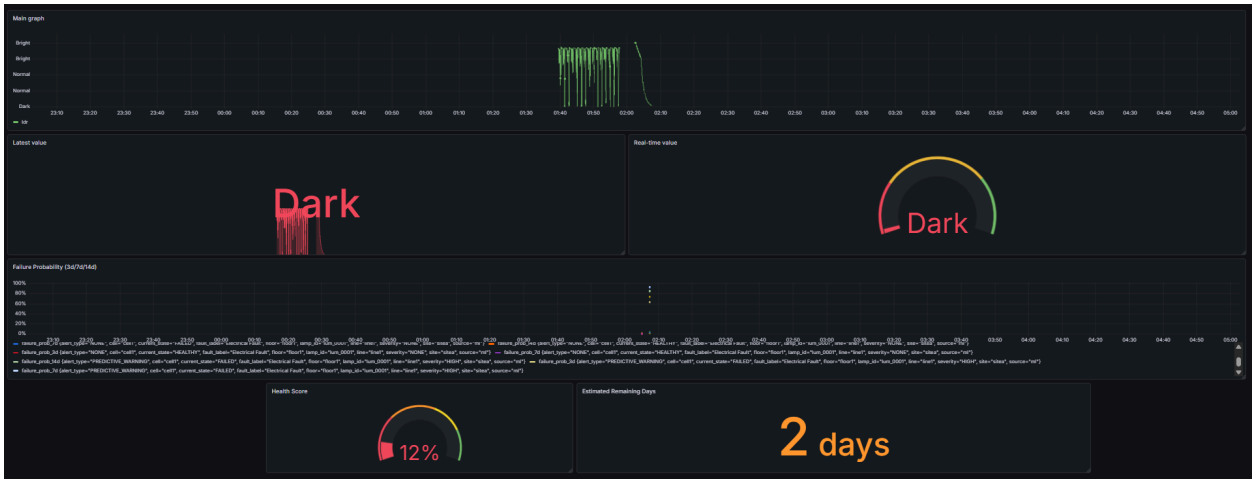
Training data for the TinyML model is generated by the software simulator under varied environmental conditions, producing approximately 100,000 labelled samples. The fault classifier is trained on the 34,000-row street_light_fault_prediction_dataset.csv dataset. The failure predictor is trained on synthetic degradation sequences generated to cover a wide range of ageing trajectories.

10. User Interface and Scalability

10.1 Centralised User Interface via Grafana Dashboards

The primary user interface is built using Grafana, which provides a powerful and highly customisable visualisation layer. Connected directly to the InfluxDB time-series database, the Grafana dashboards deliver real-time insights into the health of the entire lighting network. Multiple dashboard views are provided:

- Single Bulb Dashboard (one_bulb_dashboard): Granular, deep-dive analysis of an individual luminaire's metrics, including LDR readings, RGB colour shifts, and ripple percentage.

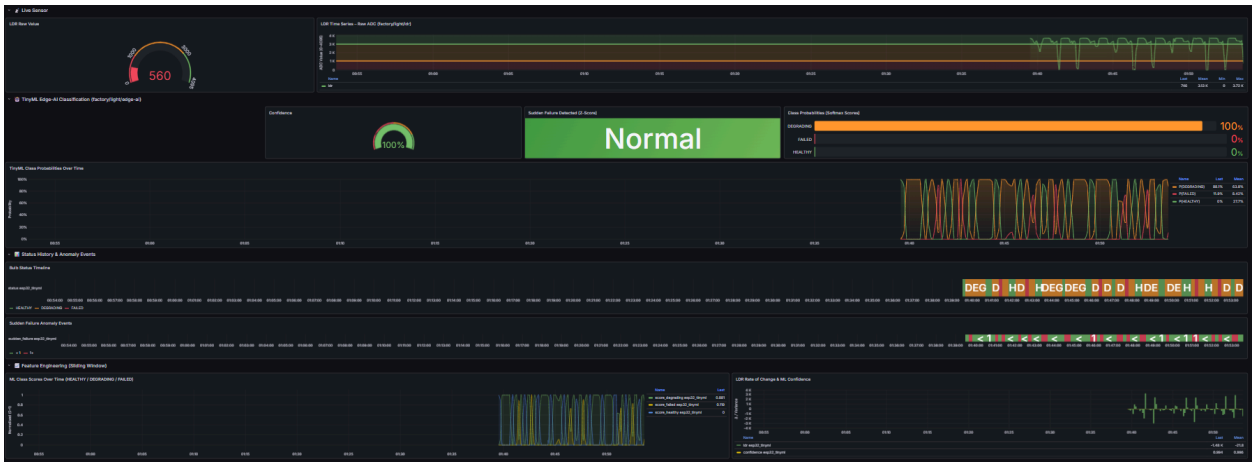


Single bulb dashboard (simulation)

- Scaled Bulb System Dashboard (scaled_bulb_system): A macro-level overview of the entire facility's lighting network, showing aggregate health states and critical alerts.

Bulb system Dashboard

- TinyML Dashboard (tinyml_dashboard): Real-time edge inference results, class confidence scores, and Z-score anomaly events.



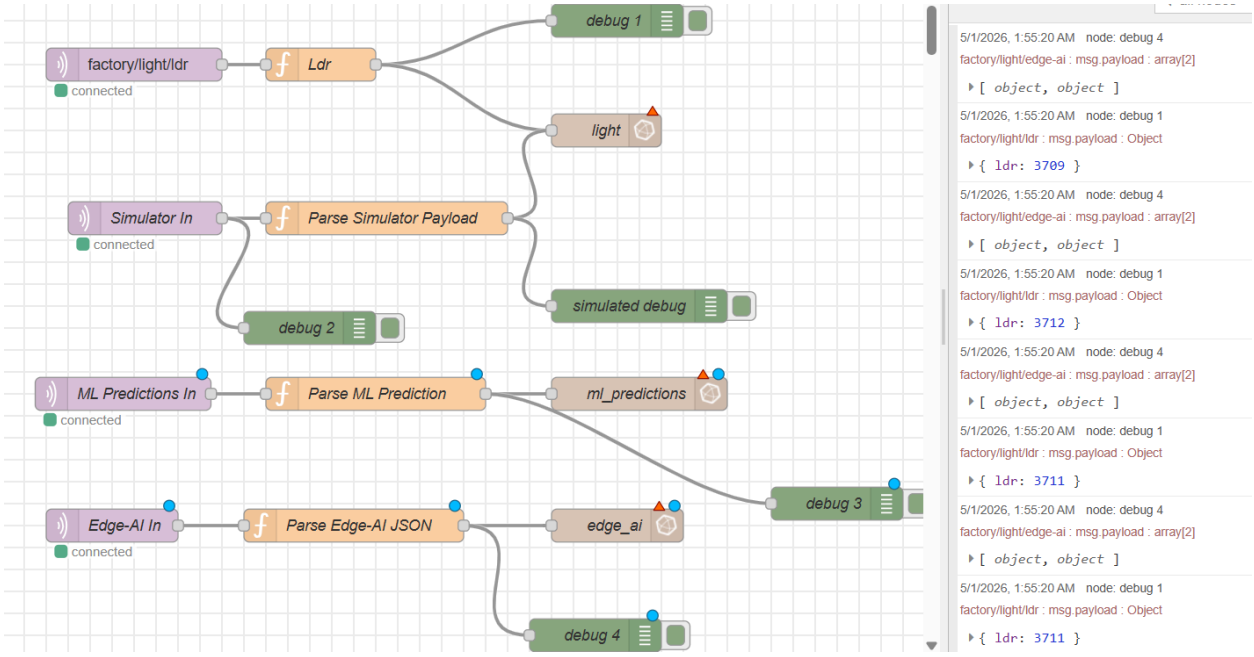
Realworld system

10.2 Intelligent Alerting System

The alerting system is driven by the outputs of the Edge AI and Cloud ML models rather than by simple threshold breaches. When the TinyML model on an ESP32 detects a statistical anomaly via Z-score calculation, or when the XGBoost cloud model predicts an imminent failure based on observed degradation trends, an alert is immediately generated. This ensures that alerts are highly accurate, contextually meaningful, and carry a low false-positive rate.

10.3 Automated Notification Pipeline

Leveraging Node-RED as the data-routing middleware, the system subscribes to specific MQTT alert topics (e.g., factory/site/alerts/status). When a critical degradation or failure event is detected, Node-RED automatically formats and dispatches notifications through configurable channels, which may include email servers, SMS gateways, or messaging platforms such as Slack or Telegram. Alerts include the physical location of the affected luminaire and its predicted remaining useful life.



Node-red

10.4 Scalability for Multiple Nodes

The system architecture is inherently designed for horizontal scalability:

- Edge Scalability: Each ESP32 node runs its own TinyML inference locally, ensuring that the central server does not become a bottleneck as thousands of luminaires are added to the network.
- Infrastructure Scalability: The entire backend is fully Dockerised, enabling the MQTT broker, InfluxDB, and AI services to be scaled across multiple servers or migrated to cloud environments (such as AWS or Azure) as data throughput increases.
- Topic Architecture: A structured, hierarchical MQTT topic naming convention (e.g., factory/area1/light01) allows Node-RED and Grafana to dynamically discover, filter, and visualise data from newly added nodes without requiring any changes to the backend code.

11. Use Cases

11.1 Predictive Maintenance in Industrial and Commercial Facilities

The primary application of this system is enabling true predictive maintenance across large-scale facilities such as factories, warehouses, and office buildings. Instead of relying on inefficient run-to-failure approaches or wasteful scheduled mass replacements, facility managers can use the Digital Twin to monitor the exact health of every individual luminaire. By detecting early signs of degradatio such as micro-flickering, electrical ripple anomalies, or colour shifts. The AI models accurately predict each luminaire's Remaining Useful Life (RUL). Maintenance teams can then order replacements in advance and schedule targeted servicing

during non-operational hours, minimising downtime and preventing safety hazards in critical areas.

11.2 Smart City Street Lamp Management

Maintaining municipal street lighting is logistically challenging and expensive, often requiring night-time patrols or relying on citizen complaints to identify outages. By integrating this Edge AI system into street lamp networks, municipalities can remotely monitor the health of thousands of lights across a city. The system can immediately detect abrupt failures caused by weather damage or electrical surges, and predict gradual degradation over time. This enables city planners to dispatch repair crews only to locations where lamps are approaching failure, dramatically reducing fuel costs, lowering carbon emissions from maintenance vehicles, and ensuring consistent visibility and safety on public roads.

11.3 Quality Assurance and Accelerated Life Testing in Manufacturing

For manufacturers of LED bulbs and lighting components, the software simulator and hardware models function as a digital testbed. During the research and development phase, engineers can use the Digital Twin to simulate years of thermal and electrical stress in a matter of hours through accelerated life testing. By comparing real prototype telemetry against the simulator's degradation physics, manufacturers can rapidly identify design flaws, improve driver circuitry, and validate the expected lifespan of their products before market release.

11.4 Energy Optimisation and Dynamic Load Balancing

As LED performance degrades, luminaires often require more current to produce the same lumen output, decreasing overall energy efficiency. The monitoring system can detect when a bulb or an entire lighting zone begins to draw excessive power relative to its light output. Building Management Systems (BMS) can utilise this data to dynamically adjust lighting loads and dimming healthier bulbs to save energy or balancing the electrical load across circuits to prevent strain ultimately lowering electricity bills and extending the operational life of the entire lighting network.

12. Contributions

E/20/032 — Bandara A.M.N.C.: Digital Twin Simulator Development

Led the architecture and implementation of the software-based Digital Twin simulation platform used to generate high-fidelity synthetic telemetry when live hardware streams were unavailable. Designed and implemented degradation physics models for LED ageing, including luminous intensity decay, RGB spectral drift, ripple growth, anomaly scoring, and remaining useful life (RUL) estimation. Built the simulator as a full-stack system with a Node.js/Express backend for real-time state evolution and control APIs, and a React/Vite frontend for interactive monitoring, fault injection, and scenario-based stress testing.

Engineered the simulator to operate as a scalable fleet of virtual luminaires rather than a single source, introducing per-instance lifecycle management, location-aware identity (site/floor/line/cell/lamp), and parallel simulation execution. Implemented structured MQTT publishing for each virtual luminaire and integrated the pipeline with Node-RED and InfluxDB by

extending ingestion flows to wildcard topic subscriptions and richer predictive payload persistence (sensor, environment, identity, and derived-health fields). Refactored backend and frontend code into modular service/component layers to improve maintainability, testability, and production readiness, and hardened container build pipelines for production deployment across varying Docker base-image environments.

E/20/157 — Janakanatha S.M.B.G.: AI Implementation (Edge and Cloud)

Janakanatha was responsible for the end-to-end intelligence of the predictive maintenance system, deploying machine learning models at both the edge and cloud levels.

- Edge AI (TinyML): Developed the lightweight neural network models capable of running directly on the resource-constrained ESP32 microcontrollers. This work involved writing the C++ inference logic — using TensorFlow Lite Micro and the EloquentTinyML library — to extract features via a sliding window approach, calculate statistical Z-scores for sudden failure detection, and classify the real-time bulb state (HEALTHY, DEGRADING, or FAILED) locally without requiring continuous internet connectivity.
- Cloud AI: Developed the centralised Python AI service that subscribes to aggregated MQTT telemetry data. This involved implementing an XGBoost-based predictive pipeline to perform heavy-duty anomaly detection, calculate the precise Remaining Useful Life (RUL) of individual luminaires, and expose these predictions via a FastAPI REST endpoint to trigger proactive maintenance alerts.

E/20/254 — Methpura S.K.P.: Hardware Integration and Dockerised Infrastructure

Methpura architected the physical hardware layer and the robust containerised infrastructure required to tie the entire system together.

- Hardware: Managed the physical deployment by writing the C++ firmware for the ESP32 devices to interface with LDR sensors, manage Wi-Fi and MQTT connectivity, and implement safe relay switching logic. Also developed the Arduino-based LED lifecycle simulator (led_sim.ino) to visually demonstrate degradation phases using PWM fading and strobing.
- Infrastructure and DevOps: Authored the comprehensive docker-compose.yml stack to orchestrate the complete backend architecture, containerising the Mosquitto MQTT broker, InfluxDB, Grafana, Node-RED, the simulator services, and the custom Python AI service into a unified local network. Designed the Node-RED data-routing flows to pipe telemetry into InfluxDB, and created the real-time Grafana dashboards used to visualise system health and ML predictions.

13. References

The following references are provided in IEEE citation format.

[1] D. S. Kumar, Md. Samer, A. Abhinav, P. Tejasree, G. S. Varsha, and D. Koushik, "Design and Implementation of an IoT-Enabled Smart Street Lighting System using STM32 Microcontroller and ESP32," 2023.

[2] Sk. M. Sorif, D. Saha, and P. Dutta, "Smart Street Light Management System with Automatic Brightness Adjustment Using Bolt IoT Platform," 2021.

[3] D. Sunehra and S. Rajasri, "Automatic Street Light Control System using Wireless Sensor Networks," 2017.

[4] S. Rai, K. Kumari, and D. Verma, "Self-Supplied Automatic Control of Street Light," 2020.

[5] S. Y. Kadirova and D. I. Kajtsanov, "A Real Time Street Lighting Control System," 2017.

[6] P. Arjun, S. Stephenraj, N. N. Kumar, and K. N. Kumar, "A Study on IoT Based Smart Street Light Systems," 2019.

Appendix: LED Degradation Modelling — Rationale for the Simulator

This appendix documents the design decisions behind the LED degradation simulator, including the choice of parameters, mathematical models, alignment with published literature, and guidance for research-grade extension.

A.1 Mathematical Models

Time Acceleration

The simulator uses accelerated ageing for practical usability. The simulated time advances as follows:

```
timeHours += TICK_SECONDS × speed × 30     (TICK_SECONDS = 0.5)
```

At a speed factor of 1, one wall-clock second advances approximately 30 simulated hours.

Intensity Model

The intensity is computed as follows and then clamped to [0.05, 1.1]:

```
intensity = exp(-0.00006 × timeHours)
          × (1 - max(0, ambientTemp - 25) × 0.0025)
          × (1 - max(0, humidity - 50) × 0.001)
          × (1 + (driveCurrent - 350) × 0.0004)
```

- The exp(-k·t) term models first-order lumen depreciation over time, consistent with LM-80/TM-21 standards.
- The temperature multiplier encodes additional stress derating above 25 °C.
- The humidity multiplier applies a further penalty above 50% relative humidity.
- The drive current term provides a luminous boost at higher currents while permitting penalty terms to dominate long-term health trends.

RGB Channel Model

Base channel values are derived from intensity with deliberate offsets to simulate a warm-white LED:

```
R = 255 × intensity × 1.00 + noise
G = 240 × intensity × 0.95 + noise
B = 225 × intensity × 0.90 + noise
C = R + G + B
```

Ripple Model

The ripple percentage is computed as follows and clamped to [0.2, 45]:

```
ripple = (1 + timeHours × 0.0001 + (driveCurrent - 350) × 0.0005) × 1.6 + noise
```

Remaining Useful Life Model

RUL is computed using a transparent linear penalty model:

```
RUL = baseLifeHours
    - timeHours × hoursPenalty
    - rippleRatio × ripplePenalty × 100
    - tempRise × tempPenalty
    - currentRiseRatio × currentPenalty × 100
    - anomalyScore × anomalyPenalty
```

A.2 Cross-Validation Against Published Standards

The exponential intensity decay term is consistent with the first-order degradation representation used throughout the LM-80/TM-21 LED reliability standard ecosystem. The temperature, humidity, and current penalty multipliers reflect multi-stress lifetime findings in which thermal and electrical stresses jointly accelerate degradation. RGB colour shift modelling aligns with DOE SSL reliability guidance identifying phosphor and package ageing as primary contributors to chromaticity drift. The ripple model's directional increase with age and current stress matches capacitor-ageing literature, where rising ESR and declining capacitance manifest as increased output ripple.

A.3 Recommendations for Research-Grade Extension

If this simulator is used to support publishable studies, the following upgrade sequence is recommended:

5. Fit the exp(-k·t) decay coefficient and stress multipliers to LM-80-measured data for the specific LED package under study.

6. Replace the fixed RGB channel multipliers with chromaticity trajectory models expressed in u'v' or CCT + Duv drift space.
7. Replace the scalar ripple model with ESR and capacitance state variables coupled to temperature-dependent capacitor ageing equations.
8. Re-estimate all RUL penalty coefficients using regression or survival models trained on historical maintenance data from the target deployment environment.